

1. Logika cyfrowa

Układ	Formuła / sygnatura	Opis
Sumatory		
Półsumator	$s = x \oplus y$ $c = x \& y$	Dodaje dwa bity. s to suma, c to bit przeniesienia (carry).
Pełny sumator (FA)	$s = x \oplus y \oplus ci$ $co = x \& y \mid ci \& (x \oplus y)$	Jak półsumator, ale przyjmuje też przeniesienie wejściowe ci .
Sumator n-bitowy	$n \times \text{FA}$	Kaskada (ripple-carry): wyjście C_i trafia na wejście kolejnego FA. Ostatnie co to Carry Out całości.

Układy kombinacyjne		
Dekoder	$n \rightarrow 2^n$	Wejście koduje liczbę k . Na wyjściu zapalony jest tylko bit nr k .
Multiplekser	$2^n + n \rightarrow 1$	2^n bitów danych + n bitów sterujących S . Na wyjście przechodzi S -ty bit danych.
ALU	$A, B, f \rightarrow C$	Dekoder na bitach f wybiera operację (np. $A + B$, $A \mid B$, $A \& B$); multipleksowanie wyników bramkami AND/OR.

Układy sekwencyjne (pamięć)		
Przerzutnik S-R	S - set, R - reset	Przechowuje 1 bit (Q). S=1 ustawi $Q = 1$, R=1 zeruje, S=R=0 trzyma stan. Dwa sprzężone NOR -y.
Sterowany poziomem	aktywny gdy CLK = 1	Wejścia S / R bramkowane AND-em z zegarem. Stan zmienia się tylko przy CLK=1 .
Sterowany zboczem	zbocze 1 → 0	Dwa przerzutniki poziomowe w kaskadzie (master-slave), drugi z zanegowanym zegarem. Stan przepisuje się w momencie zmiany zegara.

2. Typy danych

Typ	char	short	int	long	float	double	void*
x86-64	1	2	4	8	4	8	8

3. Kodowanie liczb i konwersja

$$\text{B2U}(X) = \sum_{i=0}^{w-1} x_i 2^i \quad \text{B2T}(X) = -x_{w-1} 2^{w-1} + \sum_{i=0}^{w-2} x_i 2^i$$

$$\text{T2U}(x) = x < 0 ? x + 2^w : x \quad \text{U2T}(u) = u > \text{TMax} ? u - 2^w : u$$

Skróty: **B** = bity, **U** = unsigned, **T** = ze znakiem (kod U2).
Stąd **B2U** = bity → unsigned, podobnie **U2T**, **UMax**, **TMax**, **UAdd**, **TAdd**.

Stała	Bity	Wartość
UMax	11...1	$2^w - 1$
TMax	01...1	$2^{w-1} - 1$ (INT_MAX)
TMin	10...0	-2^{w-1} (INT_MIN)
-1	11...1	te same bity co UMax

Promocja w C: **unsigned** i **int** w jednym wyrażeniu/porównaniu ($< > ==$) → **int** promowany do **unsigned** (bity bez zmian, $-1 \rightarrow \text{UMax}$).

4. Rozszerzanie, obcinanie i przesunięcia

Rozszerz. 0 (zext)	unsigned : dopisuje 0 z góry
Rozszerz. znak (sext)	signed : powiela bit znaku (MSB)
x << k	$x \cdot 2^k$ (sgn./uns.)
u >> k	$\lfloor u/2^k \rfloor$ - logiczne (zera)
x >> k	arytmetyczne (kopia MSB), zaokr. do $-\infty$
$x/2^k$ do 0	(x + (1<<k)-1) >> k
Strength reduction	x*24 → (x<<5)-(x<<3)

5. Operacje, overflow i endianness

UAdd/TAdd	$(u + v) \bmod 2^w$	
Nadmiar (+)	$u, v > 0$, a wynik < 0 (wynik -2^w)	
Nadmiar (-)	$u, v < 0$, a wynik ≥ 0 (wynik $+2^w$)	
Negacja U2	<code>-x = ~x+1 ; -TMin=TMin , -0=0</code>	
Wykr. nadmiaru (<code>s=x+y</code>)	<code>((s^x)&(s^y))>>(w-1)</code>	
Maska / abs	<code>m=x>>31; abs=(x^m)-m</code>	
Schemat	Najniższy adres	0x0123
Big Endian	MSB	[01][23]
Little Endian	LSB	[23][01]

6. Zmiennoprzecinkowe (IEEE 754)

$V = (-1)^s \times M \times 2^E$ Bias = $2^{k-1} - 1$				
Format	bity	s	exp (<i>k</i>)	frac (<i>n</i>)
float	32	1	8	23
double	64	1	11	52

Kategorie wartości

Kategoria	exp	Własności
Znormalizowane	<code>!= 0...0</code> <code>!= 1...1</code>	$E = \text{exp} - \text{Bias}$. $M = 1.\text{frac}$. Najgęściej ułożone wokół 0.
Zdenormaliz.	<code>== 0...0</code>	$E = 1 - \text{Bias}$. $M = 0.\text{frac}$. Reprezentują ± 0.0 (frac = 0) i liczby bardzo bliskie zera.
Specjalne	<code>== 1...1</code>	<code>frac == 0</code> → $\pm\infty$ (nadmiar, dzielenie przez 0). <code>frac != 0</code> → NaN (np. $\sqrt{-1}$, $\infty - \infty$).

Zaokrąglanie GRS i Round-to-Even

Domyślny tryb to **Round-to-Even** (zapobiega błędowi statycznemu przy sumowaniu).

... x x G i R S S S ...		
G - ostatni zachowany, R - pierwszy usunięty, S - OR reszty		
Warunek	Ułamek	Akcja
<code>R=0</code>	<code>< 0.5</code>	W dół (odcięcie)
<code>R=1, S=1</code>	<code>> 0.5</code>	W górę (+1 do <code>G</code>)
<code>R=1, S=0</code>	<code>= 0.5</code>	Do parzystej: w górę gdy <code>G=1</code> , w dół gdy <code>G=0</code>

Własności matematyczne i rzutowanie

Brak łączności	$(a + b) + c \neq a + (b + c)$ - zaokrąglenia
Brak rozdzielności	$a(b + c) \neq ab + ac$
<code>int</code> → <code>float</code>	możliwa utrata precyzji (zaokrągła)
<code>int</code> → <code>double</code>	bezstratne (52 > 32 bity)
<code>float/double</code> → <code>int</code>	obcina do 0; poza zakresem lub NaN → TMin (<code>0x80000000</code>)

7. Rejestry x86_64

<code>%rax</code>	<code>%eax</code> <code>%ah</code> <code>%al</code>	<code>%rbx</code>	<code>%ebx</code> <code>%bh</code> <code>%bl</code>
<code>%rcx</code>	<code>%ecx</code> <code>%ch</code> <code>%cl</code>	<code>%rdx</code>	<code>%edx</code> <code>%dh</code> <code>%dl</code>
<code>%rsi</code>	<code>%esi</code> <code>%si</code> <code>%sil</code>	<code>%rdi</code>	<code>%edi</code> <code>%di</code> <code>%dil</code>
<code>%rbp</code>	<code>%ebp</code> <code>%bp</code> <code>%bpl</code>	<code>%rsp</code>	<code>%esp</code> <code>%spl</code>
<code>%r8</code>	<code>%r8d</code> <code>%r8w</code> <code>%r8b</code>	<code>%r9</code>	<code>%r9d</code> <code>%r9w</code> <code>%r9b</code>

Uwaga: Rejestry od `%r10` do `%r15` używają schematu:

`%r10`, `%r10d`, `%r10w`, `%r10b`.

Podział ról rejestrów

<code>%rax</code>	Akumulator. Główny rejestr arytmetyczny. Przechowuje wartość zwracaną .
<code>%rdi</code> , <code>%rsi</code> , <code>%rdx</code> , <code>%rcx</code> , <code>%r8</code> , <code>%r9</code>	Kolejno: od 1. do 6. argumentu funkcji . Caller-saved.
<code>%rsp</code>	Wskaźnik stosu (SP). Wskazuje wierzchołek ramki.
<code>%rbp</code>	Wskaźnik bazy (BP). Początek ramki stosu. Callee-saved.
<code>%rbx</code> , <code>%r12</code> – <code>%r15</code>	Ogólnego przeznaczenia. Wszystkie callee-saved .
<code>%rip</code>	Wskaźnik instrukcji (Program Counter).

8. Tryby adresowania (operandy)

Tryb	Format	Obliczanie adresu
Natychmiastowy (Imm)	<code>\$Imm</code>	<code>Imm</code>
Rejestrowy (Reg)	<code>%Ra</code>	<code>%Ra</code>
Bezpośredni (Mem)	<code>Imm</code>	<code>MEMImm</code>
Pośredni (Mem)	<code>(%Rb)</code>	<code>MEM(%Rb)</code>
Z przesunięciem	<code>D(%Rb)</code>	<code>MEM(%Rb + D)</code>
Skalowany (Ind/scaled)	<code>D(%Rb, %Ri, S)</code>	<code>MEM(%Rb+%Ri*S+D)</code>
Skalowany (baseless)	<code>(, %Ri, S)</code>	<code>MEM(%Ri * S)</code>

Legenda: `Imm` / `D` to stała liczbowa, `%Ra` / `%Rb` to rejestr bazowy, `%Ri` to rejestr indeksowy, a `S` to skala (tylko: 1, 2, 4 lub 8).

9. Flagi stanu i sterowanie

ZF	Zero. Wynik to 0 (np. argumenty są równe).
SF	Sign. Wynik jest ujemny (MSB = 1).
CF	Carry. Przepełnienie dla liczb bez znaku (unsigned).
OF	Overflow. Przepełnienie dla liczb ze znakiem (signed).

Sufiksy warunkowe (dla `jX`, `setX`, `cmovX`)

Sufiks	Znaczenie
e / z	Equal / Zero (równe / wynik to 0)
ne / nz	Not Equal / Not Zero (nierówne)
s	Sign (wynik ujemny, <code>SF=1</code>)

Liczbzy ze znakiem (signed)

g / ge	Greater / Greater or Equal
l / le	Less / Less or Equal

Liczbzy bez znaku (unsigned)

a / ae	Above / Above or Equal (No Carry)
b / be	Below / Below or Equal (Carry)

Translacja struktur kontrolnych (C → ASM)

- if-else:** `jX` (skok) lub `cmovX` (liczy obie ścieżki, bez kary za predykcję). `cmovX` odpada przy drogich gałęziach, wyluskaniach (`*p`) i efektach ubocznych (`x++`).
- switch:** tablica skoków → czas $O(1)$. Skok pośredni: `jmp *.L4(,%rdi,8)`. Brak `break` ⇒ **fall-through**.
- Pętla for:** zawsze redukowana do **while**:
`init; while(cond) { body; update; }`

Wzorce asemblerowe dla pętli:

do-while	while (Jump-to-mid)	while (Guarded)
loop: Body if(T) goto loop	goto test loop: Body test: if(T) goto loop	if(!T) goto done loop: Body if(T) goto loop done:

10. Procedury i stos

Stos rośnie w dół (niższe adresy); `%rsp` wskazuje **wierzchołek** (najniższy zajęty adres). `push` zmniejsza `%rsp` o 8, `pop` zwiększa o 8.

- `call dest`: odkłada adres powrotu na stos i skacze do `dest`.
- `ret`: zdejmuje adres powrotu ze stosu i skacze pod niego.

11. Konwencja Wywoływań (System V ABI)

`%rdi` → `%rsi` → `%rdx` → `%rcx` → `%r8` → `%r9`

Argumenty 7+: Na stosie od końca. Wynik w `%rax`.

Rejestry caller-saved vs callee-saved

Caller-saved

(Wołający musi zapisać)
Mogą zostać nadpisane w funkcji.
By je zachować, caller kładzie je na stos przed `call`.

`%rax`, `%rdi`, `%rsi`,
`%rdx`, `%rcx`, `%r8` - `%r11`

Callee-saved

(Wołany musi przywrócić)
Muszą zachować stan. Callee musi zapisać je na stos i odtworzyć przed `ret`.

`%rbx`, `%rbp`, `%r12` - `%r15`

Ramka stosu

Każde wywołanie funkcji ma własną ramkę (stąd **rekurencja** działa naturalnie).

Ramka potrzebna, gdy: brak rejestrów na zmienne (spilling), lokalna tablica/struktura, lub pobrano adres zmiennej (`&`).

Prolog

```
pushq %rbp ;  
zapisz stary base ptr  
movq %rsp, %rbp ;  
nowy base ptr = rsp  
subq $32, %rsp ;  
alokacja 32 bajtów
```

Epilog

```
addq $32, %rsp ;  
zwolnij alokację  
popq %rbp ;  
przywróć base ptr  
ret ;  
skok powrotny
```

`leave`: zastępuje `movq %rbp, %rsp` + `popq %rbp` jedną instrukcją.

12. Katalog instrukcji (AT&T)

Opcode	Efekt	Opis
<code>mov S, D</code>	$D \leftarrow S$	Kopiuje wartość z S do D.
<code>lea S, D</code>	$D \leftarrow \text{addr}(S)$	Oblicza adres S (bez czytania pamięci).
<code>add S, D</code>	$D \leftarrow D + S$	Dodawanie (ustawia flagi).
<code>sub S, D</code>	$D \leftarrow D - S$	Odejmowanie (ustawia flagi).
<code>imul S, D</code>	$D \leftarrow D * S$	Mnożenie liczb ze znakiem.
<code>sal / shl k, D</code>	$D \ll= k$	Przesunięcie w lewo (mnożenie przez 2^k).
<code>sar k, D</code>	$D \gg= k$	Arytmetyczne w prawo (dzielenie). Kopiuje znak.
<code>shr k, D</code>	$D \gg= k$	Logiczne w prawo. Dopełnia zerami.
<code>and / or S, D</code>	$D \leftarrow D \& / S$	Operacje bitowe (ustawiają flagi).
<code>xor S, D</code>	$D \leftarrow D \wedge S$	Często jako <code>xor %rax, %rax</code> do zerowania.

Porównania i sterowanie

<code>cmp S1, S2</code>	$S2 - S1$	Ustawia flagi jak odejmowanie, nie zapisuje wyniku.
<code>test S1, S2</code>	$S2 \& S1$	Ustawia flagi jak AND .
<code>jX dest</code>	$\text{if}(X) \%rip \leftarrow \text{dest}$	Skok warunkowy (X = warunek).
<code>setX D</code>	$\text{if}(X) D \leftarrow 1$	Ustawia najmłodszy bajt na 0 lub 1 wg flag.
<code>cmove S, D</code>	$\text{if}(X) D \leftarrow S$	Warunkowe kopiowanie (zamiast skoku).

Stos i zarządzanie procedurami

<code>push S</code>	$\%rsp - = 8$ $M[\%rsp] \leftarrow S$	Odkłada na stos (zmniejsza <code>%rsp</code>).
<code>pop D</code>	$D \leftarrow M[\%rsp]$ $\%rsp + = 8$	Zdejmuje ze stosu do D (zwiększa <code>%rsp</code>).
<code>call dest</code>	$\text{push } \%rip$ $\%rip \leftarrow \text{dest}$	Skok do funkcji (zapisuje adres powrotu na stosie).
<code>ret</code>	$\text{pop } \%rip$	Zdejmuje adres powrotu ze stosu i skacze pod niego.
<code>leave</code>	$\%rsp \leftarrow \%rbp$ $\text{pop } \%rbp$	Sprząta ramkę stosu (odwrotność prologu).

Uwaga o sufiksach: Instrukcje przyjmują sufiks określający rozmiar danych:

b (8-bit), **w** (16-bit), **l** (32-bit), **q** (64-bit).

Np. `movl` kopiuje 32 bity (i automatycznie zeruje górną połowę 64-bitowego rejestru!).